

Centro Federal de Educação Tecnológica de Minas Gerais

Engenharia de Computação

Computação Gráfica

Relatório de Desenvolvimento

Renderização da Bandeira Brasileira em Espaço RGB

João Paulo da Cunha Faria

Maria Eduarda Teixeira Souza

Divinópolis - MG

Agosto de 2026

1 Introdução

A representação digital de imagens é fundamentada na manipulação de cores através de espaços de cor bem definidos. O espaço RGB (Red, Green, Blue) é amplamente utilizado em computação gráfica por sua adequação a dispositivos de exibição como monitores e projetores, que emitem luz através da combinação de três canais de cor primária.

Este trabalho propõe a renderização da bandeira brasileira utilizando a biblioteca Pillow (*Python Imaging Library*), explorando como primitivas gráficas simples e representações de cores em RGB podem ser combinadas para reproduzir elementos iconográficos bem conhecidos. A bandeira é composta por três camadas geométricas: um fundo verde que representa a história e a esperança, um losango amarelo que simboliza as riquezas minerais, e uma esfera azul (representação simplificada do globo terrestre) no centro.

2 Fundamentos de Cor e Representação RGB

2.1 Espaço de Cor RGB

O espaço RGB é um modelo aditivo de cores onde cada cor é representada como uma combinação de três componentes:

- **Red (Vermelho):** intensidade de 0 a 255
- **Green (Verde):** intensidade de 0 a 255
- **Blue (Azul):** intensidade de 0 a 255

Uma tupla (R, G, B) representa uma cor específica. Por exemplo:

- Verde: $(0, 148, 64)$
- Amarelo: $(255, 255, 0)$
- Azul: $(0, 0, 255)$

Cada componente pode ser visto como um canal de luz que, quando combinado com os outros, produz a cor final perceida pelo observador. A biblioteca Pillow utiliza este padrão nativamente para objetos de imagem no modo `"RGB"`.

2.2 Primitivas Gráficas

O projeto utiliza duas primitivas fundamentais:

- **Polígono:** Um polígono é definido por uma sequência de pontos no plano 2D. No contexto deste projeto, um losango é construído através de quatro pontos que definem seus vértices.
- **Elipse:** Uma elipse é definida por seu retângulo envolvente, especificado por dois pontos diagonais (superior esquerdo e inferior direito). Um círculo é um caso especial de elipse onde os eixos maior e menor possuem o mesmo comprimento.

3 Metodologia

3.1 Materiais e Configuração

O projeto foi desenvolvido em *Python*, utilizando a biblioteca `Pillow` (PIL - Python Imaging Library), instalada via:

```
pip install Pillow
```

A biblioteca Pillow fornece abstrações de alto nível para manipulação de imagens raster, permitindo criar, desenhar e salvar imagens em diversos formatos.

3.2 Código Completo

A seguir, apresenta-se a implementação completa do programa de renderização da bandeira brasileira:

Listing 1: Código completo para renderização da bandeira brasileira

```
from PIL import Image, ImageDraw

def criar_fundo_rgb(largura, altura, cor_rgb):
    """
    Cria uma imagem RGB com uma cor s lida
    """
    # Imagem.new aloca a matriz na mem ria.
    fundo = Image.new("RGB", (largura, altura), cor_rgb)

    # Retorna a imagem
    return fundo

def criar_losango_rgb(largura, altura, cor_rgb, bandeira):
    """
    Cria um losango RGB
    """

    # Ativando modo de sobreescrever pixels
    losango = ImageDraw.Draw(bandeira)

    # Coordenadas do losango
    pontoA = (largura / 2, altura * 0.05)
    pontoB = (largura * 0.95, altura / 2)
    pontoC = (largura / 2, altura * 0.95)
    pontoD = (altura * 0.05, altura / 2)

    # Lista de coordenadas para desenhar o losango
    coordenadas = [(pontoA), (pontoB), (pontoC), (pontoD)]

    # Criar o losango
    losango.polygon(coordenadas, fill=cor_rgb)

    # Retornar
    return bandeira


def criar_esfera_rgb(largura, altura, cor_rgb, bandeira):
    """
    Cria uma esfera RGB
    """

    # Ativando modo de sobreescrever pixels
    esfera = ImageDraw.Draw(bandeira)

    # Coordenadas da esfera (dois ponto, superior esquerdo e inferior direito)
    centro_x = largura / 2
    centro_y = altura / 2

    # Raio da esfera
    raio = largura * 0.20

    # Coordenadas do quadrado que envolve a esfera
    pontoA = (centro_x - raio, centro_y - raio)
    pontoB = (centro_x + raio, centro_y + raio)
```

```
55
56      # Lista de coordenadas para desenhar a esfera
57      coordenadas = [(pontoA), (pontoB)]
58
59      # Criar a esfera
60      esfera.ellipse(coordenadas, fill=cor_rgb)
61
62      # Retornar
63      return bandeira
64
65
66  if __name__ == "__main__":
67      # dimens es da imagem
68      LARGURA = 2400
69      ALTURA = 1600
70
71      # Testando a cor Verde: (0, 148, 64) RGB
72      cor_verde_rgb = (0, 148, 64)
73      print(f"Cor Verde em RGB: {cor_verde_rgb}")
74
75      # Criar o fundo da bandeira
76      bandeira = criar_fundo_rgb(LARGURA, ALTURA, cor_verde_rgb)
77
78      # Testando a cor Amarela: (255, 255, 0) RGB
79      cor_amarela_rgb = (255, 255, 0)
80      print(f"Cor Amarela em RGB: {cor_amarela_rgb}")
81
82      # Criar o losango
83      bandeira = criar_losango_rgb(LARGURA, ALTURA, cor_amarela_rgb, bandeira)
84
85      # Testando a cor Azul: (0, 0, 255) RGB
86      cor_azul_rgb = (0, 0, 255)
87      print(f"Cor Azul em RGB: {cor_azul_rgb}")
88
89      # Criar a esfera
90      bandeira = criar_esfera_rgb(LARGURA, ALTURA, cor_azul_rgb, bandeira)
91
92      # Salvar a imagem
93      bandeira.save("bandeira.png")
```

3.3 Análise Detalhada do Código

3.3.1 Importações e Bibliotecas

O programa inicia com a importação da biblioteca Pillow:

```
1  from PIL import Image, ImageDraw
```

Dois componentes são importados:

- `Image`: fornece a classe responsável por criar, carregar e manipular imagens raster
- `ImageDraw`: fornece ferramentas para desenhar primitivas gráficas (linhas, polígonos, elipses, etc.) sobre imagens existentes

3.3.2 Função `criar_fundo_rgb`

Esta função é responsável pela criação do fundo da bandeira:

```python
def criar_fundo_rgb(largura, altura, cor_rgb):
    fundo = Image.new("RGB", (largura, altura), cor_rgb)
    return fundo
```

Detalhamento:

- **Parâmetros:** recebe largura, altura e uma tupla RGB
- **Image.new():** aloca uma nova imagem em memória no modo `"RGB"` com as dimensões especificadas
- **cor_rgb:** define a cor inicial de todos os pixels da imagem
- **Retorno:** devolve o objeto `Image` criado para posterior manipulação

No contexto deste projeto, esta função é chamada com `cor_verde_rgb = (0, 148, 64)`, criando uma imagem 600×400 pixels totalmente preenchida com o verde da bandeira brasileira.

3.3.3 Função `criar_losango_rgb`

Esta função desenha o losango amarelo sobre a bandeira:

```python
def criar_losango_rgb(largura, altura, cor_rgb, bandeira):
    losango = ImageDraw.Draw(bandeira)
    pontoA = (largura / 2, altura * 0.05)
    pontoB = (largura * 0.95, altura / 2)
    pontoC = (largura / 2, altura * 0.95)
    pontoD = (altura * 0.05, altura / 2)
    coordenadas = [(pontoA), (pontoB), (pontoC), (pontoD)]
    losango.polygon(coordenadas, fill=cor_rgb)
    return bandeira
```

Detalhamento linha a linha:

- `ImageDraw.Draw(bandeira)`: cria um contexto de desenho vinculado à imagem passada como argumento. Este objeto permite executar operações gráficas sobre a imagem
- `pontoA = (largura / 2, altura * 0.05)`: define o vértice superior do losango, posicionado no topo centro da imagem (50% de largura, 5% de altura)
- `pontoB = (largura * 0.95, altura / 2)`: define o vértice direito do losango, posicionado no lado direito (95% de largura, 50% de altura)
- `pontoC = (largura / 2, altura * 0.95)`: define o vértice inferior do losango, posicionado no fundo centro (50% de largura, 95% de altura)
- `pontoD = (altura * 0.05, altura / 2)`: define o vértice esquerdo do losango (5% de largura, 50% de altura). Nota: há um erro nesta linha — deveria ser `(largura * 0.05)` ao invés de `(altura * 0.05)`, mas o código foi preservado como está na implementação original
- `coordenadas = [...]`: agrupa os quatro vértices em uma lista para passar ao método de desenho
- `losango.polygon(coordenadas, fill=cor_rgb)`: preenche o polígono definido pelos vértices com a cor especificada (amarelo)
- `return bandeira`: devolve a imagem modificada para posterior processamento

3.3.4 Função `criar_esfera_rgb`

Esta função desenha o círculo azul centralizado:

```
def criar_esfera_rgb(largura, altura, cor_rgb, bandeira):
    esfera = ImageDraw.Draw(bandeira)
    centro_x = largura / 2
    centro_y = altura / 2
    raio = largura * 0.20
    pontoA = (centro_x - raio, centro_y - raio)
    pontoB = (centro_x + raio, centro_y + raio)
    coordenadas = [(pontoA), (pontoB)]
    esfera.ellipse(coordenadas, fill=cor_rgb)
    return bandeira
```

Detalhamento linha a linha:

- `esfera = ImageDraw.Draw(bandeira)`: cria um novo contexto de desenho sobre a imagem
- `centro_x = largura / 2`: calcula a coordenada X do centro da imagem (300 pixels para largura 600)
- `centro_y = altura / 2`: calcula a coordenada Y do centro da imagem (200 pixels para altura 400)
- `raio = largura * 0.20`: define o raio do círculo como 20% da largura (120 pixels)
- `pontoA = (centro_x - raio, centro_y - raio)`: calcula o canto superior esquerdo do retângulo envolvente do círculo (180, 80)
- `pontoB = (centro_x + raio, centro_y + raio)`: calcula o canto inferior direito do retângulo envolvente (420, 320)
- `coordenadas = [(pontoA), (pontoB)]`: agrupa os dois pontos diagonais
- `esfera.ellipse(coordenadas, fill=cor_rgb)`: desenha uma elipse preenchida. Como os dois pontos diagonais possuem a mesma diferença em X e Y, cria-se um círculo perfeito preenchido com azul
- `return bandeira`: devolve a imagem modificada

3.3.5 Bloco Principal

O bloco `if __name__ == "__main__":` executa o fluxo de renderização quando o script é executado diretamente:

```
LARGURA = 2400
ALTURA = 1600
cor_verde_rgb = (0, 148, 64)
print(f"Cor Verde em RGB: {cor_verde_rgb}")
bandeira = criar_fundo_rgb(LARGURA, ALTURA, cor_verde_rgb)
```

Define as dimensões da tela (600×400 pixels), especifica a cor verde e cria a imagem de fundo:

```
cor_amarela_rgb = (255, 255, 0)
print(f"Cor Amarela em RGB: {cor_amarela_rgb}")
bandeira = criar_losango_rgb(LARGURA, ALTURA, cor_amarela_rgb, bandeira)
```

Define a cor amarela e desenha o losango sobre o fundo já criado:

```
cor_azul_rgb = (0, 0, 255)
print(f"Cor Azul em RGB: {cor_azul_rgb}")
bandeira = criar_esfera_rgb(LARGURA, ALTURA, cor_azul_rgb, bandeira)
```

Define a cor azul e desenha o círculo sobre a bandeira:

```
bandeira.save("bandeira.png")
```

Finalmente, salva a imagem resultante em formato PNG no diretório de trabalho atual.

4 Resultados e Análises

Durante a execução do programa principal, o sistema executa as seguintes operações em sequência:

1. Cria uma imagem de 600×400 pixels com fundo verde RGB $(0, 148, 64)$
2. Desenha um losango amarelo RGB $(255, 255, 0)$ sobre o fundo verde
3. Desenha uma esfera azul RGB $(0, 0, 255)$ no centro da composição
4. Salva a imagem final em formato PNG

A saída da console exibe as cores em representação RGB:

```
Cor Verde em RGB: (0, 148, 64)
Cor Amarela em RGB: (255, 255, 0)
Cor Azul em RGB: (0, 0, 255)
```

A renderização visual resultante pode ser conferida na Figura 1, que apresenta a bandeira brasileira renderizada através de primitivas gráficas básicas. A composição em camadas demonstra como primitivas simples (polígono e elipse) combinadas com a manipulação precisa de cores RGB podem reproduzir artefatos visuais reconhecíveis.



Figura 1: Renderização final da bandeira brasileira, composta por um fundo verde, um losango amarelo e uma esfera azul centralizada.

5 Conclusão

O projeto demonstrou com sucesso a utilização da biblioteca Pillow para renderização de imagens através de primitivas gráficas e manipulação direta de cores no espaço RGB. A implementação confirmou que conceitos fundamentais de geometria computacional — posicionamento de vértices, cálculo de retângulos envolventes e preenchimento de formas — são suficientes para a composição de imagens reconhecíveis.

A abordagem modular adotada permite que cada camada da bandeira seja construída independentemente, facilitando ajustes de cores, dimensões e geometrias. Este trabalho reforça a importância da abstração entre modelos matemáticos de primitivas gráficas e sua implementação em bibliotecas de alto nível como Pillow.

Referências

[1] RODRIGO, Raulivan. **Fundamento de Cores: Computação Gráfica**. Notas de Aula (06 - Funda de Cores.pdf). Departamento de Computação, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), 2026.

[2] CLARK, Alex; CONTRIBUTORS. **Pillow (PIL Fork) Documentation**. Disponível em: https://pillow.readthedocs.io/. Acesso em: Agosto de 2026.